



SAP HANA Cloud, SAP HANA Database Knowledge Graph Engine Guide

Generated on: 2026-05-24 12:05:59 GMT+0000

SAP HANA Cloud, SAP HANA Database | QRC 1/2026

Public

Original content: https://help.sap.com/docs/HANA_CLOUD_DATABASE/5c3488c559d64d2c8b4f331a1d9b281a?locale=en-US&state=PRODUCTION&version=2026_1_QRC

Warning

This document has been generated from SAP Help Portal and is an incomplete version of the official SAP product documentation. The information included in custom documentation may not reflect the arrangement of topics in SAP Help Portal, and may be missing important aspects and/or correlations to other topics. For this reason, it is not for production use.

For more information, please visit <https://help.sap.com/docs/disclaimer>.

SAP HANA Cloud, SAP HANA Database Knowledge Graph Engine Guide

SAP HANA Cloud, SAP HANA database contains the SAP HANA Cloud knowledge graph engine.

The SAP HANA Cloud knowledge graph engine is an integral part of SAP HANA Cloud. It is an addition to the SAP HANA Cloud multi model engines. It is designed for processing large-scale graph data and executing complex graph queries with high efficiency using knowledge graphs. SAP HANA Cloud knowledge graph engine enables organizations to gain insights from their graph datasets, discover patterns, perform advanced graph analytics and unlock the value of interconnected data.

The SAP HANA Cloud knowledge graph engine enables working with knowledge graphs. It is built using World Wide Web Consortium (W3C) web standards specifications of graph data called RDF (containing triples with the format subject-predicate-object) and its associated W3C web standards query language called SPARQL (see [Technical Concepts](#)). Knowledge graphs are stored in a triple store.

SPARQL-SQL-Interoperability and SPARQL Query Execution

The `SPARQL_EXECUTE ( )` stored procedure is an interfacing SQL procedure that executes RDF queries (both SPARQL Query Forms and SPARQL Update Forms). `SPARQL_TABLE ( )` and `SQL_TABLE ( )` are functions that enable "interoperability" between SQL and SPARQL. These functions enable SQL queries to embed SPARQL and SPARQL queries to embed SQL to federate query capabilities.

SAP HANA Cloud knowledge graph engine, SPARQL, and SAP HANA Cloud, SAP HANA database SQL are integrated with the following SQL constructs:

SQL Construct	Description
SPARQL_EXECUTE stored procedure	Allows you to execute all types of SPARQL statements (both SPARQL Query Forms and SPARQL Update Forms).
SPARQL_TABLE function	Embeds SPARQL SELECT queries within SQL statements and returns an SQL table as a result.
SQL_TABLE function	Embeds SQL queries directly within a SPARQL statement and federates the queries to the SQL engine resulting in SQL statements being converted into graph patterns.

Related Information

[Integration](#)

[SPARQL Query Forms](#)

[SPARQL Update Forms](#)

[Connectivity Interfaces](#)

Getting Started

Information you need to consider before you start working with the SAP HANA Cloud knowledge graph engine.

[Architecture](#)

The SAP HANA Cloud knowledge graph engine is a dedicated store in SAP HANA Cloud that stores and processes graph data.

[Memory Consumption and Sizing Considerations](#)

It is important to understand the memory usage and sizing considerations for the SAP HANA Cloud knowledge graph engine to optimize performance and resource allocation for the database.

[Business Benefits and Use Cases](#)

Knowledge graphs in SAP HANA Cloud enable enhanced data contextualization and AI performance through multi model data integration. You would use this to build intelligent applications, improve generative AI outcomes, and make better data-driven decisions across your enterprise.

[Technical Concepts](#)

RDF (Resource Description Framework) and SPARQL (SPARQL Protocol and RDF Query Language) enable representation and querying data in a knowledge graph. RDF uses triples to represent information, while SPARQL is a query language for RDF data. Ontologies define the framework for knowledge graphs.

[Enable Triple Store](#)

To access the SAP HANA Cloud knowledge graph engine, you must enable the **Triple Store** option.

[DEFAULT Graph and Named Graphs](#)

SAP HANA Cloud knowledge graph engine supports a DEFAULT graph and named graphs.

[Knowledge Graph Engine Transaction Processing](#)

SAP HANA Cloud knowledge graph engine is tightly integrated with SAP HANA database to provide the experience of a single transaction across other engines such as column store, document database, and so on.

Architecture

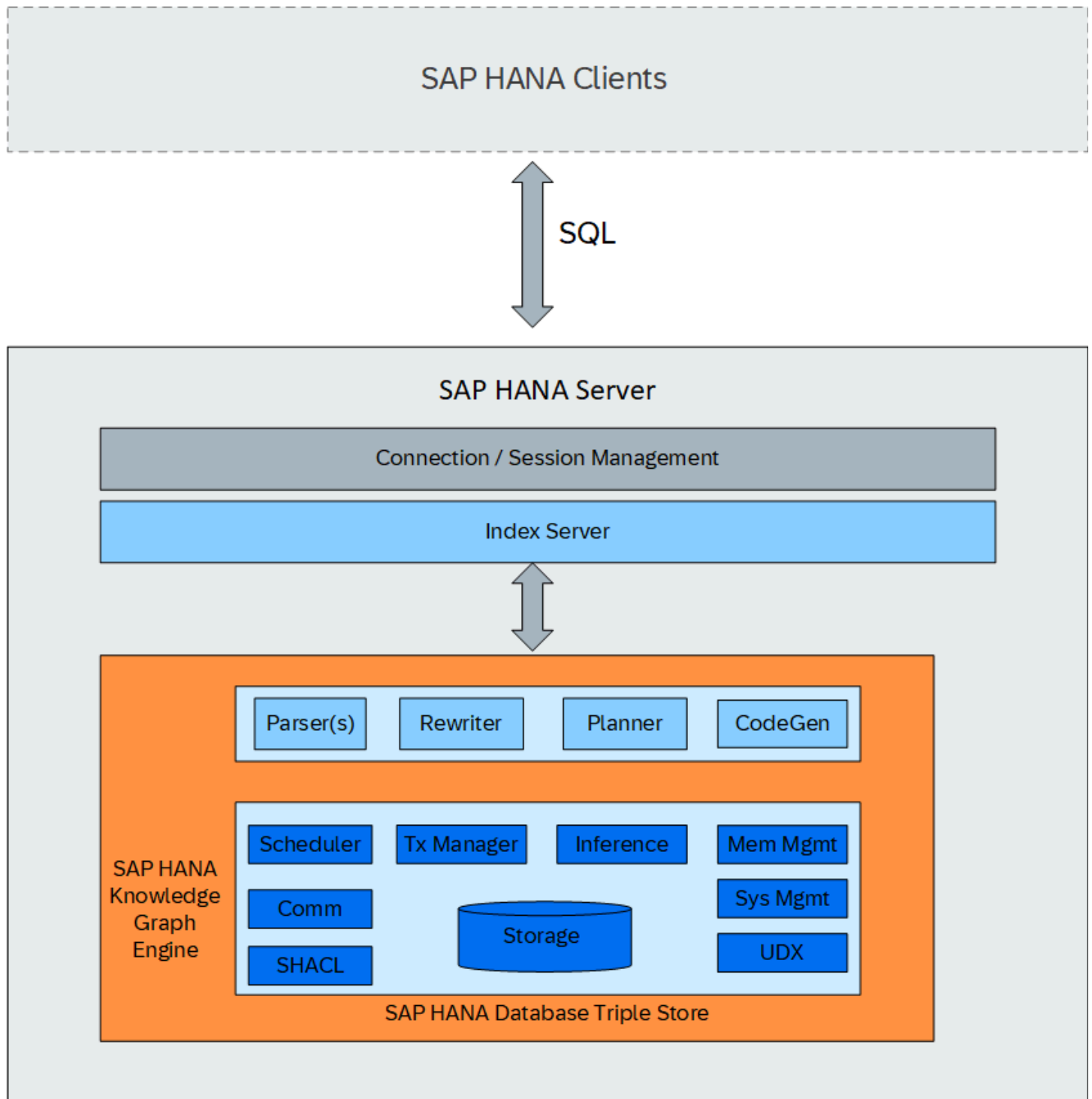
The SAP HANA Cloud knowledge graph engine is a dedicated store in SAP HANA Cloud that stores and processes graph data.

The SAP HANA Cloud knowledge graph engine is a graph database system specifically built using W3C web standards specifications. It stores graph data that is represented in RDF (Resource Description Framework), also known as triples, and it queries the data by using a language called SPARQL (SPARQL Protocol and RDF Query Language).

SAP HANA Cloud knowledge graph engine is a high performance graph analytics engine, which is built utilizing the Massively Parallel Processing (MPP) concepts to leverage the power of parallel processing and distributed computing.

It is specifically optimized for handling large RDF datasets and its associated graph-based structures and employs query optimization techniques to enable fast and scalable processing of SPARQL queries.

The SAP HANA Cloud knowledge graph engine runs on a single compute node. Each compute node contains slices, which are parallel computes within a compute node that take advantage of multi core CPUs for parallel processing. Even though the system is running on one compute node, the knowledge graph engine takes advantage of multi core CPUs and executes queries in MPP fashion. Data is "sharded" and loaded into slices, so each slice is essentially a mini-compute within a compute node.



Related Information

[Technical Concepts](#)

[Terminology](#)

Memory Consumption and Sizing Considerations

It is important to understand the memory usage and sizing considerations for the SAP HANA Cloud knowledge graph engine to optimize performance and resource allocation for the database.

Memory usage varies significantly depending on the makeup of the data, such as the data types and sizes of literal values, and the complexity of the queries that you run. Data is loaded as triples, and the storage required for each triple ranges anywhere from 12 bytes for the most basic triple to 1 megabyte for a triple that stores pages of text from an unstructured document.

You need to consider the memory consumption and sizing for both the SAP HANA Cloud knowledge graph engine and the SAP HANA Cloud instance (see [SAP HANA Database Size](#)).

SAP HANA Cloud knowledge graph engine uses 50% of the allocated memory for data and 50% for query processing. For example, if 40% of the total memory of the SAP HANA instance is assumed to be used for SAP HANA Cloud knowledge graph engine, then 20% of the total memory is assigned to data usage of SAP HANA Cloud knowledge graph engine.

Caution

Currently there is no enforcement of limits on memory resources.

A minimum of 8 vCPUs for SAP HANA Cloud knowledge graph engine is recommended. But, for better performance, more vCPUs are recommended.

Business Benefits and Use Cases

Knowledge graphs in SAP HANA Cloud enable enhanced data contextualization and AI performance through multi model data integration. You would use this to build intelligent applications, improve generative AI outcomes, and make better data-driven decisions across your enterprise.

- **Enhanced data contextualization**

RDF graphs organize and represent complex relationships within data, enabling businesses to embed domain-specific knowledge directly into applications. This improves decision-making by providing contextually relevant insights.

- **Improved AI and LLM performance**

By grounding large language models (LLMs) with knowledge graphs, businesses can ensure that AI-powered applications deliver more accurate and context-aware responses, enhancing the user experience.

- **Streamlined data integration**

Knowledge graphs facilitate the seamless integration of diverse data sources, breaking down silos and enabling a unified view of business data across the organization.

- **Enhanced user experience**

Knowledge graphs make AI interfaces more intuitive, allowing users to interact naturally with complex data systems, thereby improving user engagement and satisfaction.

- **The multi model advantage**

The ability to handle both structured and unstructured data within a single platform allows businesses to scale their applications easily while maintaining flexibility in data processing and analysis.

- **Enhance your intelligent data applications and generative AI**

The in-memory database capabilities of SAP HANA Cloud provide high-performance data processing. The addition of SAP HANA Cloud knowledge graph engine enables applications to gain an extra layer of understanding. SAP HANA Cloud knowledge graph engine is particularly valuable when it comes to enabling Gen AI use cases, such as grounding large language models in domain-specific knowledge. By storing and retrieving semantic facts about a business's operations, relationships, and data, the knowledge graph can provide LLMs with the context they need to deliver more accurate and relevant responses, ultimately improving the quality of human-computer interaction.

- **Establish a data fabric**

With the SAP HANA Cloud knowledge graph engine working seamlessly alongside the other multi model engines in SAP HANA Cloud, such as the relational data, spatial, graph, JSON document store, and vector engines, businesses can bring

structured and unstructured data together in a single platform, establishing a data fabric. This unified approach allows companies to query and analyze data holistically, breaking down traditional data silos. The semantic linking of data creates a unified view, allowing for an accurate contextual understanding of complex concepts, such as the term "material" in different scenarios like manufacturing, sales, or procurement.

- **Improve decision making and logical inference**

Knowledge graphs contribute to better data governance by establishing clearer semantical relationships and rules for data entities, which improves logical reasoning and helps users make more informed, accurate decisions with fewer errors.

Technical Concepts

RDF (Resource Description Framework) and SPARQL (SPARQL Protocol and RDF Query Language) enable representation and querying data in a knowledge graph. RDF uses triples to represent information, while SPARQL is a query language for RDF data. Ontologies define the framework for knowledge graphs.

For a list of World Wide Web Consortium (W3C) recommendations, see [SAP HANA Cloud, SAP HANA Database SPARQL Reference Guide](#).

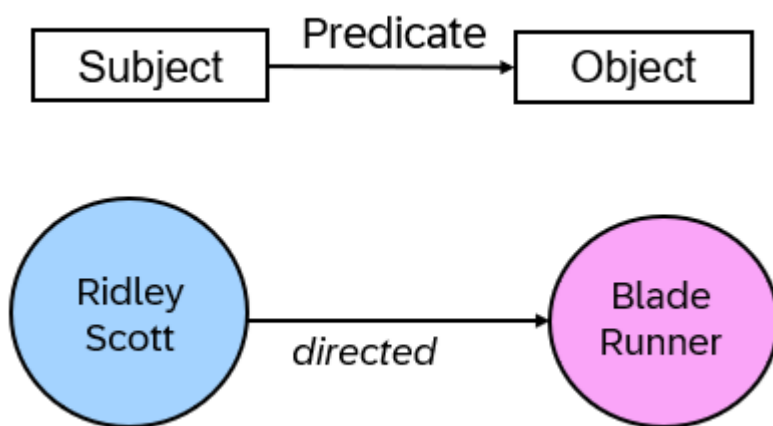
For additional terms on knowledge graphs and SPARQL, see [Terminology](#).

Knowledge Graph

A knowledge graph is a representation of data, enhanced by real-world information. The knowledge graph captures relationships between entities, such as people, products, or events, and connects them in a meaningful way. A knowledge graph organizes information as triples where facts are represented as subject, predicate, and object. These triples form the core of the knowledge graph, revealing the logical relationships between different entities.

Knowledge graphs are typically used to model complex relationships between different entities, such as people, places, and concepts, and can be used to power applications such as search engines, recommendation systems, and data analysis tools.

For more information about the following example, see [Tutorial](#).



RDF (Resource Description Framework)

RDF works by representing information using triples, which consist of the following components:

- **Subject:** Represents the resource being described.
- **Predicate:** Represents a property or attribute of the subject.

- Object: Represents the value or target of the property.

The following triples describe that "Ridley Scott directed the film Blade Runner." These triples are taken from the knowledge graph example provided in this guide (see [Tutorial](#)):

```
<http://www.wikidata.org/entity/Q184843> <http://www.w3.org/1999/02/22-rdf-syntax-ns#type> <http://www.wikidata.org/entity/Q184843> <http://kg.demo.sap.com/title> "Blade Runner" .
<http://www.wikidata.org/entity/Q56005> <http://www.w3.org/1999/02/22-rdf-syntax-ns#type> <http://www.wikidata.org/entity/Q56005> <http://kg.demo.sap.com/directed> <http://www.wikidata.org/entity/Q56005> <http://www.w3.org/2000/01/rdf-schema#label> "Ridley Scott"
```

SPARQL (SPARQL Protocol and RDF Query Language)

SPARQL is a query language designed specifically for querying RDF data. It provides a powerful and expressive syntax for formulating queries that retrieve and manipulate information from RDF knowledge graphs.

When a SPARQL query is executed, it traverses the RDF graph, matching the specified patterns and applying any conditions or filters. The query engine retrieves the relevant triples that satisfy the query criteria and returns the results in a tabular format.

Triple

A triple in RDF is a data structure that consists of three components: subject, predicate, and object. The subject is the resource being described. Predicates describe the relationship between the subject and the object. Objects are values or resources related to the subject through the predicate. All three components are represented by Uniform Resource Identifiers (URIs).

For example, in the triple "Tom Hanks acted in Forrest Gump", "Tom Hanks" is the subject, "acted in" is the predicate, and "Forrest Gump" is the object.

Turtle

The Turtle data format is a way of representing data using the RDF. It's a way of serializing RDF data in a human-readable and easy-to-write format. Turtle is defined by the W3C and uses a syntax like the N-Triples format, with added support for prefixes and shorthand notations to make it more concise and readable. Turtle is commonly used for writing and sharing RDF data on the web.

Ontology and Web Ontology Language (OWL)

An ontology is a guiding framework used to define the facts that occur in a domain and how those higher-level descriptions should relate to each other. Instead of focusing on the facts, an ontology regulates the resulting knowledge graph. For example, in the context of movies, an ontology could define the concept of "movies" and its attributes like title, director, genre, and release date, making the framework flexible and scalable for future data. Ontologies act as the blueprint for knowledge graphs, structuring and connecting data to allow deeper insights and reasoning.

Web Ontology Language (OWL) is used to represent the ontology of knowledge graphs.

Enable Triple Store

To access the SAP HANA Cloud knowledge graph engine, you must enable the **Triple Store** option.

The **Triple Store** option can be enabled while creating an instances. For existing instances, **Triple Store** can be enabled on the **Advanced Settings** tab under **Manage Configuration**.

For more information about database instances, see [Creating SAP HANA Database Instances](#) and [Managing SAP HANA Database Instances](#).

For more information about the **Triple Store** option, see [SAP HANA Database Additional Features](#).

DEFAULT Graph and Named Graphs

SAP HANA Cloud knowledge graph engine supports a DEFAULT graph and named graphs.

DEFAULT Graph

The built-in DEFAULT graph (an unnamed graph) is always present and accessible for all users with PUBLIC access privileges. The DEFAULT graph is readable and writable by all users. Triples stored in the DEFAULT graph can be accessed with FROM DEFAULT in the SPARQL statements. Any data inserted into the knowledge graph without specifying a name for the graph is inserted into the DEFAULT graph.

The following SQL console example inserts triples into the DEFAULT graph:

```
CALL SPARQL_EXECUTE('
  INSERT DATA {
    <P1> a <Person>; <name> "Joe Bud"; <email> "joe.bud@example.com" . }
', '', '?', '?');
```

The following SQL console example selects all triples from the DEFAULT graph:

```
SELECT * FROM SPARQL_TABLE('
  SELECT ?s ?p ?o
  FROM DEFAULT
  WHERE { ?s ?p ?o . }
  ORDER BY ?s
');
```

Named Graphs

With **named graphs**, triples can be stored according to their meaning. With FROM or FROM NAMED, queries can specify any combination of named graphs together with the DEFAULT graph to create an RDF-merge of the graphs to operate on.

Use the GRANT statement to grant permissions to a named graph.

The following SQL console example inserts triples into the named graph john_movies:

```
CALL SPARQL_EXECUTE('
  INSERT DATA { GRAPH <john_movies> {
    <P1> a <Director>; <name> "Martin Scorsese".
    <P2> a <Director>; <name> "Steven Spielberg".
    <P3> a <Director>; <name> "Quentin Tarantino".
    <P4> a <Director>; <name> "Woody Allen". }
  }
', '', '?', '?');
```

The following SQL console example selects all triples from the given named graphs:

```
SELECT * FROM SPARQL_TABLE('
  SELECT ?s ?p ?o
  FROM <john_movies>
```



```
FROM <kgdocu_movies>
WHERE { ?s ?p ?o . }
');
```

The following SQL console example selects all triples from the DEFAULT graph and all named graphs:

```
SELECT * FROM SPARQL_TABLE( '
SELECT ?s ?p ?o
WHERE { ?s ?p ?o . }
');
```

The following SQL console example selects all triples from all named graphs and adds the name of the graphs to the output:

```
SELECT * FROM SPARQL_TABLE( '
SELECT ?s ?p ?o ?g
WHERE { GRAPH ?g { ?s ?p ?o . } }
');
```

Related Information

[SPARQL_EXECUTE Stored Procedure](#)

[SPARQL_TABLE Function](#)

[GRANT Update Form](#)

[View Knowledge Graphs in Database Objects](#)

Knowledge Graph Engine Transaction Processing

SAP HANA Cloud knowledge graph engine is tightly integrated with SAP HANA database to provide the experience of a single transaction across other engines such as column store, document database, and so on.

Interoperability with SAP HANA Database Objects

You can use the same transaction to perform updates to a knowledge graph and other SAP HANA database objects such as tables, views, and so on. Executing a single COMMIT or ROLLBACK command for a transaction guarantees atomic commit behavior across SAP Knowledge Graph and other SAP HANA database objects.

Example

1. Disable AUTOCOMMIT mode for the session. For example, if you're using the SAP HANA HDBSQL command-line tool, you can do this using the -z option described [here](#). If you're using the SQL console from SAP HANA Database Explorer, you can do this by editing the connection settings as described [here](#).
2. Create a column store table called T1 and insert data into it :

```
CREATE COLUMN TABLE T1 (ID INT);
```

3. Insert data into knowledge graph <MYGRAPH> and table T1, then run a COMMIT.

```
INSERT INTO T1 VALUES(1);
```

```
CALL SPARQL_EXECUTE( '
INSERT DATA {
  GRAPH <MYGRAPH> {
```

```

        <P1> A <PERSON>; <NAME> "JOHN"; <AGE> 32 .
    }
}', ' ', ?, ?);

COMMIT;

```

Inserting the data into a knowledge graph automatically creates it, if it doesn't already exist.

4. Roll back the transaction:

```

INSERT INTO T1 VALUES(2);
CALL SPARQL_EXECUTE('
INSERT DATA {
    GRAPH <MYGRAPH> {
        <P2> A <PERSON>; <NAME> "SMITH"; <AGE> 35 .
    }
}', ' ', ?, ?);

ROLLBACK;

```

This rolls back the transaction on both the column store table and the knowledge graph.

5. Run a select statement on table T1:

```
SELECT * FROM T1;
```

This should show committed transactions only:

ID
1

6. Run a select on MYGRAPH:

```

SELECT * FROM SPARQL_TABLE('
    SELECT ?S ?P ?O
    FROM <MYGRAPH>
    WHERE {
        ?S ?P ?O
    }
');

```

This should show committed transactions only:

S	P	O
P1	http://www.w3.org/1999/02/22-rdf-syntax-ns#type	PERSON
P1	NAME	JOHN
P1	AGE	32

Isolation Levels

SAP HANA Cloud knowledge graph engine is fully transactional and allows only committed data to be read by a transaction. It has the following default isolation levels which can't be changed:

- **READ COMMITTED**- for read-only transactions
- **REPEATABLE READ**- for write-transactions

For complete information on these levels and what they provide, see [SET TRANSACTION Statement \(Transaction Management\)](#).

Example

In this example, there is a read transaction on `connection2` and a write transaction on `connection3`. The read transaction on `connection2` consistently sees all committed data, ensuring **READ COMMITTED** isolation. Meanwhile, the write transaction on `connection3` can only view the data that was committed before it initiated a local edit, providing **REPEATABLE READ** isolation.

- Disable AUTOCOMMIT mode for the session. For example, if you're using the SAP HANA HDBSQL command-line tool, you can do this using the `-z` option described [here](#). If you're using the SQL console from SAP HANA Database Explorer, you can do this by editing the connection settings as described [here](#).

Connection1	Insert data and execute a COMMIT: <pre>CALL SPARQL_EXECUTE(' INSERT DATA { GRAPH MYGRAPH { <P1> A PERSON; NAME "JOHN"; } }', '', '?', '?'); COMMIT;</pre>
Connection2	Run a select statement to show the first row: <pre>SELECT * FROM SPARQL_TABLE(' SELECT ?S ?P ?O FROM <MYGRAPH> WHERE { ?S ?P ?O } ');</pre>
Connection3	Run a select statement to show the first row: <pre>SELECT * FROM SPARQL_TABLE(' SELECT ?S ?P ?O FROM <MYGRAPH> WHERE { ?S ?P ?O } ');</pre> Insert data to create a local edit for this transaction:

	<pre>CALL SPARQL_EXECUTE(' INSERT DATA { GRAPH <MYGRAPH> { <P2> A <PERSON>; <NAME> "SMITH"; } }', '', '?', '?');</pre>
Connection1	Insert data and then run a COMMIT: <pre>CALL SPARQL_EXECUTE(' INSERT DATA { GRAPH <MYGRAPH> { <P3> A <PERSON>; <NAME> "ELON"; } }', '', '?', '?); COMMIT;</pre>
Connection2	Run a select statement to show the first and third rows: <pre>SELECT * FROM SPARQL_TABLE(' SELECT ?S ?P ?O FROM <MYGRAPH> WHERE { ?S ?P ?O } ');</pre>
Connection3	Run a select statement to show the first and second rows: <pre>SELECT * FROM SPARQL_TABLE(' SELECT ?S ?P ?O FROM <MYGRAPH> WHERE { ?S ?P ?O } ');</pre>

Transactional Savepoints

A savepoint is a location to which a transaction can return if part of the transaction is conditionally canceled. You can create one using the SAP HANA Cloud, SAP HANA database [SAVEPOINT statement](#).

SAP HANA Cloud knowledge graph engine supports rolling back a transaction to the named savepoint without terminating the transaction. To do this, use the SAP HANA database [ROLLBACK TO SAVEPOINT statement](#).

To release a specified savepoint, use the SAP HANA database [RELEASE SAVEPOINT statement](#).

Example

1. Disable AUTOCOMMIT mode for the session. For example, if you're using the SAP HANA HDBSQL command-line tool, you can do this using the -z option described [here](#). If you're using the SQL console from SAP HANA Database Explorer, you can do this by editing the connection settings as described [here](#).

2. Insert some triples in graph <MYGRAPH>:

```
CALL SPARQL_EXECUTE( '
INSERT DATA {
  GRAPH <MYGRAPH> {
    <P1> A <PERSON>; <NAME> "JOHN"; <AGE> 32 .
  }
}', '', '?', ?);
```

3. Set a savepoint:

```
SAVEPOINT SAVE1;
```

4. Insert some triples in graph <MYGRAPH>:

```
CALL SPARQL_EXECUTE( '
INSERT DATA {
  GRAPH <MYGRAPH> {
    <P2> A <PERSON>; <NAME> "SMITH"; <AGE> 34 .
  }
}', '', '?', ?);
```

5. Run a select statement to show all the inserted triples:

```
SELECT * FROM SPARQL_TABLE( '
SELECT ?S ?P ?O
FROM <MYGRAPH>
WHERE {
  ?S ?P ?O
}
');
```

6. Roll back the current transaction to savepoint save1:

```
ROLLBACK TO SAVEPOINT SAVE1;
```

7. Run a select statement on graph <MYGRAPH> to show triples inserted before savepoint save1 only:

```
SELECT * FROM SPARQL_TABLE( '
SELECT ?S ?P ?O
FROM <MYGRAPH>
WHERE {
  ?S ?P ?O
}
');
```

Autocommit Requirements

The AUTOCOMMIT setting does not impact any of the following operations:

- [INSERT Update Form](#)
- [INSERT DATA Update Form](#)

- [DELETE Update Form](#)
- [DELETE DATA Update Form](#)
- [CLEAR Update Form](#)
- [COPY Update Form](#)
- [CREATE INFERENCES Update Form](#)
- [LOAD Update Form](#)
- [ASK Query Form](#)
- [CONSTRUCT Query Form](#)
- [DESCRIBE Query Form](#)
- [SELECT Query Form](#)

However, when executing the operations below, the AUTOCOMMIT setting must be enabled or the session fails:

- [CREATE GRAPH Form](#)
- [DROP GRAPH Update Form](#)
- [GRANT Update Form](#)
- [REVOKE Update Form](#)
- [VACUUM Update Form](#)

SAP HANA Cloud knowledge graph engine does not support SAP HANA Cloud, SAP HANA database's [SET TRANSACTION AUTOCOMMIT DDL Statement \(Transaction Management\)](#).

Catalog Object Locks

Modifications to catalog objects are protected by object-level locks. The DROP, GRANT and REVOKE operations use an exclusive lock on catalog objects while all other SAP HANA Cloud knowledge graph engine operations use a shared lock.

In scenarios where two or more transactions are executing concurrently and referencing the same objects, it is possible that locking these objects may result in a deadlock. This can happen if the objects referenced in the concurrent transactions are in different order. To resolve the deadlock, SAP HANA Cloud knowledge graph engine would cancel one or more transactions.

Backup and Recovery

SAP HANA Cloud knowledge graph engine uses SAP HANA Cloud, SAP HANA database's transactions and data volumes to persist graph data, ensuring persistence updates are atomic and durable. It supports all backup and recovery mechanisms that are supported by SAP HANA Cloud, SAP HANA database such as recovery to a selected point in time, recreating an instance, or database instance snapshots.

See [Backups and Recovery from Backup](#) to learn more.

System Replication

SAP HANA Cloud knowledge graph engine uses SAP HANA Cloud, SAP HANA database synchronous and asynchronous replication features to maximize availability. SAP Knowledge Graph data is copied alongside all other SAP HANA Cloud data to each replica, enabling near-zero-downtime operations for instances where the SAP HANA Cloud knowledge graph engine is enabled.

See the following topics and tutorial for more information:

- [Increasing System Availability](#)
- [Upgrading Without a Restart \(Near Zero Downtime Upgrades\)](#)
- [Increasing the Resilience of a SAP HANA Cloud](#) 

Security and Authentication

To access the SAP HANA Cloud knowledge graph engine and to perform certain operations, you must be a valid and active SAP HANA database user.

For more information about user management, see the [SAP HANA Cloud, SAP HANA Database Security Guide](#).

Authentication

SAP HANA Cloud knowledge graph engine uses SAP HANA database as a gateway. All connections to the SAP HANA Cloud knowledge graph engine go through the SAP HANA client. SAP HANA database login credentials are used in the `dbapi.connect()` API call to connect to the SAP HANA database. For more information, see [Python Interface](#).

Encryption of Persisted Data

The SAP HANA Cloud knowledge graph engine implements the persistence of in-memory database by using SAP HANA Cloud persistence. Data that is persisted using SAP HANA Cloud persistence is encrypted using the data encryption scheme. For more information, see [Data Encryption](#).

Related Information

[Architecture](#)

[Enable Triple Store](#)

[Authorization](#)

Authorization

The ability to access and manage named graphs within SAP HANA Cloud knowledge graph engine is controlled by two sets of privileges. One set is defined by the SAP HANA database and the other is defined by the SAP HANA Cloud knowledge graph engine.

SAP HANA Database System Privileges

All actions that can be performed on a named graph fall into one of two categories: query and update. Each action has an assigned SAP HANA database system privilege that controls whether a user can perform the action.

- SPARQL QUERY system privilege
- SPARQL UPDATE system privilege

Users that are granted the SAP HANA database SPARQL QUERY system privilege can query named graphs, but they cannot modify or drop a named graph. Users that are granted the SAP HANA database SPARQL UPDATE system privilege can create,

modify, or drop existing named graphs, but they cannot query named graphs. Users granted both system privileges can create, modify, drop, and query named graphs.

For more information, see [SAP HANA Database System Privileges Specific to the SAP HANA Cloud Knowledge Graph Engine](#).

SAP HANA Cloud Knowledge Graph Engine Privileges

Once granted the applicable system privilege, queries and updates are then limited to named graphs that you own or to the DEFAULT graph. To query or update a named graph owned by another user, you must be explicitly granted the additional privileges defined within the SAP HANA Cloud knowledge graph engine by the owner of the named graph. These additional privileges are:

- SELECT privilege
- UPDATE privilege
- DROP privilege

SAP HANA database system privileges should be granted before SAP HANA Cloud knowledge graph engine privileges.

For more information, see [Knowledge Graph Engine Privileges Required to Query and Manage Graphs](#).

SAP HANA Database System Privileges Specific to the SAP HANA Cloud Knowledge Graph Engine

The SAP HANA database includes a set of system privileges that must be granted to SAP HANA database users before they can access any named graph, including objects they own, or the DEFAULT graph.

! Restriction

For SAP HANA databases created before SAP HANA Cloud QRC 3/2025 with the **Triple Store** option enabled and then upgraded to SAP HANA Cloud QRC 3/2025 or later, existing SAP HANA database users will no longer have access to existing knowledge graphs, regardless of whether they own the graph. One or more of the new SAP HANA database system privileges must be granted before users can access knowledge graphs.

The system privilege required depends on the task you are executing:

SAP HANA Database Privilege	Description
SPARQL QUERY system privilege	Allows users to query named graphs.
SPARQL UPDATE system privilege	Allows users to update named graphs.
SPARQL ADMIN system privilege	Allows user to execute the VACUUM update form. Unlike other update forms, VACUUM applies to all named graphs within the SAP HANA Cloud knowledge graph engine, not specific named graphs.

Use the SAP HANA database GRANT and REVOKE statements to manage access.

Related Information

- [SAP HANA Database GRANT Statement \(Access Control\)](#)
- [SAP HANA Database REVOKE Statement \(Access Control\)](#)

Knowledge Graph Engine Privileges Required to Query and Manage Graphs

SAP HANA Cloud knowledge graph engine has a set of privileges that must be granted before SAP HANA database users can query and update specific named graphs. This is in addition to the required SAP HANA database system privileges.

Only the owner of an SAP HANA Cloud knowledge graph engine named graph can see and manage the object unless privilege has been explicitly granted to another SAP HANA database user, including the SAP HANA database system administrator. All SAP HANA database users have access to the DEFAULT graph.

Privilege to these objects is granted by wrapping the SPARQL GRANT and REVOKE Update Forms within the SAP HANA database SPARQL_EXECUTE procedure.

For more information about granting these privileges, see [SPARQL Privileges](#).

Related Information

[SAP HANA Database System Privileges Specific to the SAP HANA Cloud Knowledge Graph Engine](#)

SAP HANA Database Restricted Users (Deprecated)

An SAP HANA database restricted user has read-only access to SAP HANA Cloud knowledge graph engine named graphs.

! Restriction

For SAP HANA database instances created prior to SAP HANA Cloud QRC 3/2025 with knowledge graph enabled and then upgraded to SAP HANA Cloud QRC 3/2025 or later, SAP HANA database restricted users will no longer have access to SAP HANA Cloud knowledge graph engine named graphs. Going forward, access to SAP HANA Cloud knowledge graph engine will be controlled by three SAP HANA database system privileges. See [SAP HANA Database System Privileges Specific to the SAP HANA Cloud Knowledge Graph Engine](#).

After upgrading, for existing SAP HANA database restricted users to be able to access SAP HANA Cloud knowledge graph engine they must be granted the applicable SAP HANA database system privileges. Any SPARQL privileges previously granted remain in force. Alternately, restricted users should be dropped, re-created as regular SAP HANA database users and then granted the applicable SAP HANA database system privileges and SPARQL privileges to access and manage SAP HANA Cloud knowledge graph engine named graphs.

Tutorial: Configuring Permissions Workflow

Use this workflow to help you understand how to configure user access to SAP HANA Cloud knowledge graph engine named graphs.

Privileges are granted in two stages: General access and named graph specific access.

Stage One:

The first stage controls whether a user can access the SAP HANA Cloud knowledge graph engine.

Task	Privilege Required
Create or modify a named graph.	SAP HANA database SPARQL UPDATE system privilege.

Task	Privilege Required
Query a named graph.	SAP HANA database SPARQL QUERY system privilege.

This access is granted by SAP HANA database. To grant these privileges, the grantor must have been granted these privileges with the **WITH ADMIN OPTION**.

Stage Two:

The second stage controls whether a user can modify and query named graphs that they do not own. If a user only needs to modify and query their own named graphs, then no additional privileges are required. But if a user needs to modify or query a named graph that is owned by another user, then explicitly granted privileges on the named graph are required.

Task	Privilege Required
Add or remove data from a named graph.	SAP HANA Cloud knowledge graph engine UPDATE privilege on the named graph
Query a named graph.	SAP HANA Cloud knowledge graph engine QUERY privilege on the named graph.
Drop a named graph.	SAP HANA Cloud knowledge graph engine DROP privilege on the named graph.

This access is granted by SAP HANA Cloud knowledge graph engine. The grant can only be granted by the owner of the named graph and applies to the specified named graph only.

The tasks a particular user needs to perform within the SAP HANA Cloud knowledge graph engine determines which combination of SAP HANA database system privileges and SAP HANA Cloud knowledge graph engine privileges are required. When configuring permissions, the workflow is as follows:

Identify Privileges

1. Identify the SAP HANA database users need to access the SAP HANA Cloud knowledge graph engine.
2. Identify the tasks each identified user will perform within the SAP HANA Cloud knowledge graph engine.
3. Identify the applicable SAP HANA database system privilege associated with each task.

Grant Privileges

1. Grant the identified SAP HANA database system privileges to each user.
2. Identify which named graphs will be accessed by a user other than its owner.
3. Have each identified graph owner grant the necessary SAP HANA Cloud knowledge graph engine privileges on the named graph to the identified non-owner user.

Verify the Grants

Spot check the grants to verify that they are correctly granted.

Consider the following scenario:

- USER1, USER2, and USER3 are SAP HANA database users who need to use the SAP HANA Cloud knowledge graph engine.
- USER1 needs to create and query named graph graph1.
- USER2 needs to create and query graph2 and query, but not update, graph1.

- USER3 needs to query, but not update, **graph1**. USER3 will not be allowed to query **graph2**.

Using this scenario, let's walk through the process to grant the necessary privileges for USER1, USER2, and USER3.

Identify the Privileges

1. Identify the users that requires access to the SAP HANA Cloud knowledge graph engine.
USER1, USER2, and USER3 need access.
2. Identify what tasks USER1, USER2, and USER3 need to perform in the SAP HANA Cloud knowledge graph engine.
 - a. USER1 and USER2 need to create and query named graphs.
 - b. USER3 needs to query named graphs but not create them.
3. Identify the applicable SAP HANA database system privileges associated with each task.
 - a. USER1 and USER2 need to create named graphs, so they need the SAP HANA database SPARQL UPDATE system privilege.
 - b. USER1, USER2, and USER3 need to query named graphs, so they need the SAP HANA database SPARQL QUERY system privilege.

Grant the Privileges

1. Grant the applicable SAP HANA database system privileges to each user.
 - a. Connect as a user granted the SAP HANA database SPARQL QUERY and SPARQL UPDATE system privileges with the WITH ADMIN OPTION. Grant privileges to USER1, USER2, and USER2.

```
GRANT SPARQL UPDATE, SPARQL QUERY TO USER1;
```

```
GRANT SPARQL UPDATE, SPARQL QUERY TO USER2;
```

```
GRANT SPARQL QUERY TO USER3;
```
2. Identify which users need access to specific named graphs owned by others.
 - a. USER1 needs to create, update, and query **graph1**. Since USER1 owns **graph1**, no additional privileges on **graph1** are required.
 - b. USER2 needs to query but not update **graph1** and create **graph2**. USER2 will need the SAP HANA Cloud knowledge graph engine SELECT privilege on **graph1**. Since USER2 owns **graph2**, no additional privileges on **graph2** are required.
3. Have the applicable users create the identified named graphs and then grant the applicable privileges on the named graph to the identified users.
 - a. Connected as USER1, create **graph1**.

```
CALL SPARQL_EXECUTE('
INSERT DATA {GRAPH <graph1>
{
  <P1> a <Puppet>; <name> "Cookie Monster"; <show> "Sesame Street".
```

```

    }
  }, ' ', '?', ?);

```

b. Grant the SAP HANA Cloud knowledge graph engine SELECT privilege on graph1 to USER2 and USER3.

```
CALL SPARQL_EXECUTE('GRANT SELECT ON <graph1> TO <USER2> <USER3>', ' ', '?', ?);
```

Verify the Grants

Now that all the necessary privileges are granted, let's verify the grants are correct.

We'll start with USER1. Since USER1 is able to create graph1, we know that the SPARQL UPDATE grant was successful. To verify that USER1 can query named graphs, execute:

```

SELECT * FROM SPARQL_TABLE( '
  SELECT ?s ?p ?o
  WHERE { GRAPH <graph1> { ?s ?p ?o . } }
  ORDER BY ?s ');

```

The query successfully returns the following output, which verifies that USER1 has the correct privilege to query named graphs.

s	p	o
P1	http://www.w3.org/1999/02/22-rdf-syntax-ns#type	Puppet
P1	name	Cookie Monster
P1	show	Sesame Street

Now, let's look at USER2. Connect as USER2.

USER2 should be able to query graph1:

```

SELECT * FROM SPARQL_TABLE( '
  SELECT ?s ?p ?o
  WHERE { GRAPH <graph1> { ?s ?p ?o . } }
  ORDER BY ?s ');

```

The query returns data, which indicates the query privilege is granted:

s	p	o
P1	http://www.w3.org/1999/02/22-rdf-syntax-ns#type	Puppet
P1	name	Cookie Monster
P1	show	Sesame Street

USER2 should not be able to update graph1:

```

CALL SPARQL_EXECUTE( '
  INSERT DATA {GRAPH <graph1>
  {
    <P3> a <Puppet>; <name> "Cookie Monster"; <show> "Sesame Street".
  }

```

```

    }
  }, '', ?, ?);

```

The error message confirms that USER2 does not have the privilege to update graph1.

User does not have access - UPDATE graph1

Finally, USER2 needs to create and query graph2:

```

CALL SPARQL_EXECUTE('
INSERT DATA {GRAPH <graph2>
{
  <P1> a <Puppet>; <name> "Big Bird"; <show> "Sesame Street".
}
}', '', ?, ?);

```

```

SELECT * FROM SPARQL_TABLE('
SELECT ?s ?p ?o
WHERE { GRAPH <graph2> { ?s ?p ?o . } }
ORDER BY ?s');

```

The output indicates success:

s	p	o
P1	http://www.w3.org/1999/02/22-rdf-syntax-ns#type	Puppet
P1	name	Big Bird
P1	show	Sesame Street

Lastly, let's verify USER3. Connect as USER3.

USER3 should be able to query graph1.

```

SELECT * FROM SPARQL_TABLE('
SELECT ?s ?p ?o
WHERE { GRAPH <graph1> { ?s ?p ?o . } }
ORDER BY ?s');

```

s	p	o
P1	http://www.w3.org/1999/02/22-rdf-syntax-ns#type	Puppet
P1	name	Cookie Monster
P1	show	Sesame Street

USER3 should not be able to query graph2.

```

SELECT * FROM SPARQL_TABLE('
SELECT ?s ?p ?o
WHERE { GRAPH <graph2> { ?s ?p ?o . } }
ORDER BY ?s');

```

The error message confirms that USER3 does not have the privilege to update graph2.

User does not have access - SELECT graph2

Connectivity Interfaces

The SAP HANA Cloud knowledge graph engine supports SPARQL (SPARQL Protocol and RDF Query Language), the specialized query language for RDF knowledge graph data, while tightly integrating it with SQL.

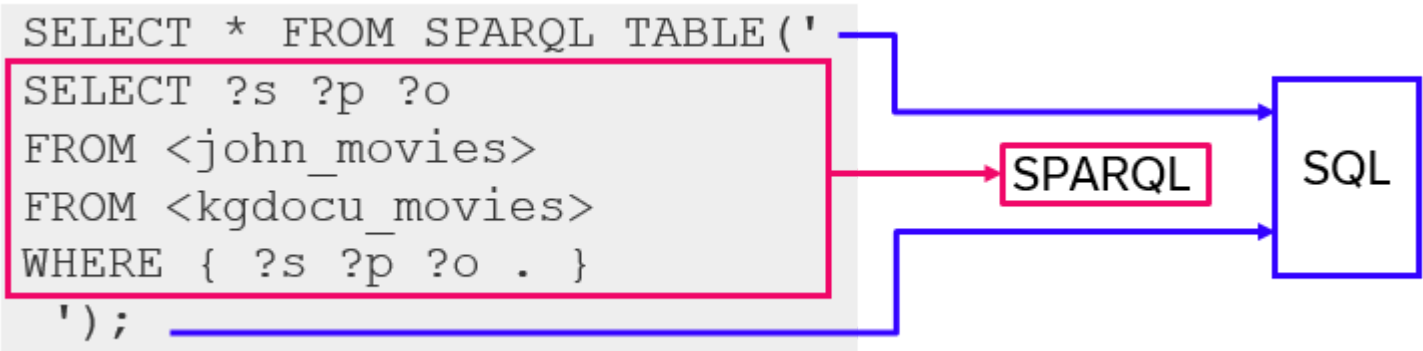
Interoperability is the ability of computer systems or software to exchange and make use of information. The SAP HANA Cloud knowledge graph engine has SQL and SPARQL interoperability to federate queries from the SQL engine or SAP HANA Cloud vector engine to the SAP HANA Cloud knowledge graph engine. It also works in the other direction, from the SAP HANA Cloud knowledge graph engine to the SQL engine.

For more information about the vector engine, see the [SAP HANA Cloud, SAP HANA Database Vector Engine Guide](#).

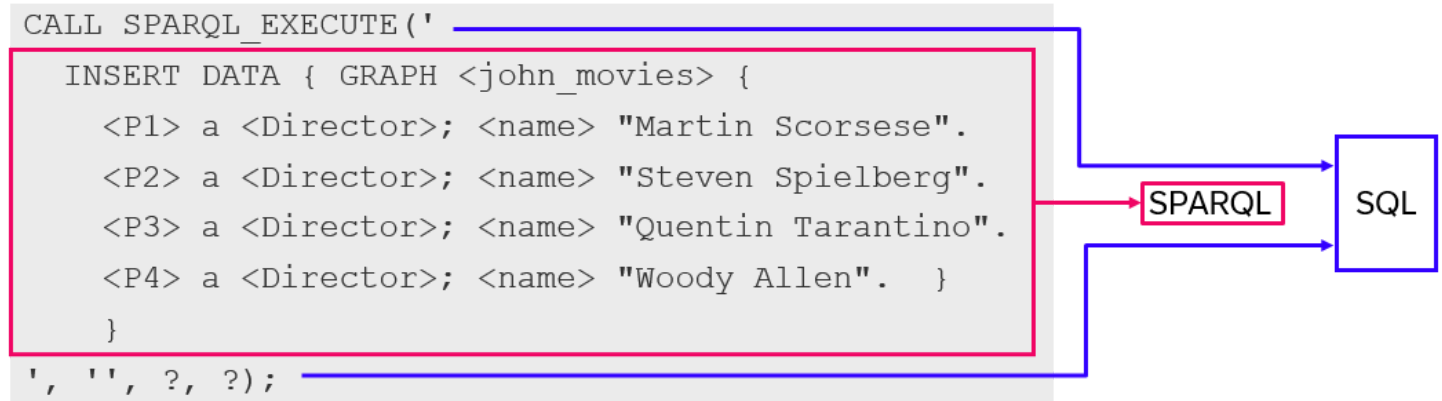
You can use the following functions and procedures:

Function or Procedure Name	Description
SPARQL_EXECUTE stored procedure	A stored procedure that executes all types of SPARQL statements, including SPARQL Query Forms and SPARQL Update Forms. Results are returned in the SPARQL query results format of XML, JSON, CSV, or TSV. For more information about result formats defined by the W3C specification, see https://www.w3.org/TR/2013/REC-sparql11-query-20130321/ .
SPARQL_TABLE function	A SQL table function that embeds SPARQL SELECT queries within SQL statements, returning results in a table format that can be processed further using SQL operations like filters, joins, and aggregations.
SQL_TABLE function	A SPARQL function that embeds SQL queries directly within SPARQL statements. This function federates queries to the SQL engine (SAP HANA index server process) and converts the results of the SQL statement into graph patterns. SQL projections are mapped to corresponding SPARQL variables, enabling smooth integration between SQL and graph data.

The following image shows an example of the structure of a statement containing the SQL function SPARQL_TABLE:



The following image shows an example of the structure of a statement containing the stored procedure SPARQL_EXECUTE:



Related Information

[SPARQL Update Forms](#)

[Integration](#)

[SPARQL Query Forms](#)

SQL Interface

The SAP HANA Cloud knowledge graph engine uses the existing SAP HANA Connectivity Interface for connecting and authenticating users and clients to the SAP HANA system and then uses the SQL interface as a gateway to send SPARQL Query and SPARQL Update statements to the SAP HANA Cloud knowledge graph engine.

To tightly integrate SAP HANA Cloud knowledge graph engine and the SAP HANA relational database, SAP HANA Cloud knowledge graph engine uses a `SPARQL_TABLE` function to execute SPARQL queries to be embedded in an SQL statement. This returns an Internal-Table-Object for further analysis by the SQL using filters, joins, aggregations, and other functions on that table.

You execute SAP HANA Cloud knowledge graph engine SQL functions and procedures and analyze their results by using the SQL console in SAP HANA Cloud Central or the SAP HANA HDBSQL command-line interface. Be aware that all statements need to adhere to the SAP HANA SQL transactional model.

Related Information

[Open the SQL Console in SAP HANA Cloud Central](#)

[SAP HANA HDBSQL \(Command-Line Reference\)](#)

[SPARQL Update Forms](#)

Python Interface

Use the Python interface to connect to an SAP HANA database and execute SPARQL queries and update options.

SAP HANA database supports multiple connectivity interfaces, including Python, Node.js, Java, and so on, that allow you to develop your own stand-alone applications. All these interfaces use drivers like JDBC or ODBC to connect to SAP HANA Cloud and can provide a higher-level API to use with the programming language of your choice. While you can use these interfaces, we recommend using the SQL console in SAP HANA Cloud Central or the SAP HANA HDBSQL command-line interface.

The Python command-line interface takes your SPARQL query, wraps and packages it as SQL, and then runs the SQL statement.

For information about installing and connecting to the Python client interface, see [Python Application Programming](#) and the [SAP HANA Client Interface Programming Reference](#).

Example

```
from hdbcli import dbapi

# connect to database using username/password
conn = dbapi.connect(user='db-username', password='db-user-password', address='database host', port=3000)

# call stored procedure to execute SPARQL Query
resp = conn.cursor().callproc('SPARQL_EXECUTE', ('SPARQL Query or RDF Turtle Data', 'Metadata headers'))
# resp[3] --> OUT: SAP HANA Cloud knowledge graph engine Response Metadata/Headers
# resp[2] --> OUT: SAP HANA Cloud knowledge graph engine Response
# resp[1] --> IN: Metadata headers describing Input and/or Output
# resp[0] --> IN: SPARQL Query or RDF Turtle Data
```

Related Information

[SQL Interface](#)

[LangChain](#)

SPARQL Query Language Reference

SPARQL query language best practices and reference information, including built-in function syntax can be found in the [SAP HANA Cloud, SAP HANA Database SPARQL Reference Guide](#).

Integration

SAP HANA Cloud knowledge graph engine runs within SAP HANA Cloud and works alongside SQL, spatial, document store, vector, and graph engines.

The SAP HANA Cloud platform empowers developers and data teams to create smarter, more context-aware AI solutions directly on operational data. SAP HANA Cloud offers support for various data types, enabling efficient storage and processing in a single platform to enhance AI workloads.

- **Relational Data for Structured Operations**

Relational data remains a cornerstone for structured operations and analytics. SAP HANA Cloud ensures that you can perform complex queries and analytics efficiently, leveraging the power of structured data.

- **Text and Spatial Data for Real-World Context**

Text and spatial data provide context to real-world scenarios, enabling you to analyze geographical information and textual content effectively. This integration supports applications ranging from location-based services to sentiment analysis.

- **Vector Data for Semantic and Similarity Search**

Vector data is crucial for semantic and similarity searches, allowing you to find patterns and relationships within large datasets. This capability enhances the accuracy of search results, making it easier to derive meaningful insights.

- **Graph Data for Relationship Modeling**

Graph data is essential for modeling explicit relationships and constructing knowledge graphs. Use the SAP HANA Cloud knowledge graph engine to help you visualize complex connections between data points, facilitating better decision-making and strategic planning.

Related Information

Comparing SAP HANA Cloud Property Graph Engine and SAP HANA Cloud Knowledge Graph Engine

The SAP HANA Cloud knowledge graph engine and the SAP HANA Cloud property graph engine work on different kinds of graphs. Both engines store and analyze connected data, but they use distinct structures to achieve this.

Use this information to help select the appropriate graph database technology for specific use cases, optimizing performance and functionality.

Property Graph

Property graphs show the relationships between entities. In a property graph, data is represented as vertices (entities) and edges (relationships), where each edge can hold different properties that describe the connection between vertices. Property graphs are stored in SQL tables.

In a supply chain context, a property graph models entities as vertices (products, suppliers, stores) and relationships as edges (supply links, delivery routes). Each vertex and edge can have multiple properties. The vertices are Product A, Supplier X, Store Y. The edges are "Supplier X supplies Product A" and "Product A is delivered to Store Y." Each edge can hold various properties like the distance of a shipment (for example, 100 km), the delivery time (for example, 2 days), and the cost of a transaction (for example, \$500).

In this example, a property graph allows you to:

- Identify the shortest delivery routes between suppliers and stores.
- Calculate the total cost of supply chain for a given product.
- Find high-reliability suppliers based on delivery time properties.

Use the property graph engine in scenarios where:

- You need graph algorithms such as shortest path, centrality, or clustering.
- Your focus is on analyzing structural relationships, not meaning or semantics.
- You want to perform fast traversals over large networks of connected data.
- Your graph data is relational in nature, such as data stored in SQL tables or JSON.

For more information, see [SAP HANA Cloud Property Graph Engine Reference](#).

Knowledge Graph

Knowledge graphs focus on establishing facts and the logical relationships between those facts. They use a subject-predicate-object structure (triples), where each fact is stored as a relationship. Knowledge graphs are stored in the triple store in RDF format.

In a supply chain context, a knowledge graph captures facts such as "Product A is made from recycled materials," "Supplier X follows sustainable sourcing practices," and "Store Y is located in an eco-friendly area."

A knowledge graph goes beyond simply showing who supplies a product. It allows you to:

- Discover which suppliers with eco-friendly practices provide products to stores in green-certified regions.
- Assess sustainability across the supply chain based on combined facts about materials and sourcing.

Use the knowledge graph engine in scenarios where:

- You need semantic reasoning or integration across diverse datasets.
- Your queries are contextual and based on meaning, not just structure.
- You want to enforce or leverage ontologies and linked vocabularies.
- You are working with federated data, possibly involving external sources.
- You are building applications with GenAI and domain-specific context.

Using the SAP HANA Cloud Vector Engine with the SAP HANA Cloud Knowledge Graph Engine

The combination of the SAP HANA Cloud vector engine and the SAP HANA Cloud knowledge graph engine enables developers to create intelligent, context-aware queries that surpass traditional keyword matching. This integration is useful for enhancing search capabilities by providing both similarity insights and explicit relationship understanding.

The vector engine is great for finding matches based on similarities, but the knowledge graph engine adds the ability to understand relationships and connections within your data.

To gain more control over the prompting results of a Large Language Model (LLM) in your application, you can leverage your own specific documents or data using Retrieval Augmented Generation (RAG).

The benefits of RAG using vector searches (pure RAG) include the following:

- It works with unstructured data and extracts information from documents, emails, reports, and logs.
- It converts text into embeddings through chunking and vectorization.
- It is effective when the required context is within a few related documents, and semantic similarity can infer relevant answers.
- The large language model (LLM) extracts facts to generate responses.
- It references original documents but lacks structured reasoning.

For more information, see [SAP HANA Database Vector Engine Guide](#).

The benefits of RAG using the vector engine with the knowledge graph engine (hybrid RAG) include the following:

- It stores structured and linked data, including entities, relationships, and hierarchies.
- Knowledge graphs are created from tables, documents, and other sources.
- Graph queries enable deep context retrieval. The required context is spread across multiple sources, making exact facts, relationships, and historical data accessible.
- The large language model (LLM) aggregates facts to generate responses.

- It provides more context, explainability, and complex reasoning by leveraging structured knowledge along with unstructured data.

For information about how to perform a multi model query, see [Multi Model Query Example](#).

Related Information

[Seamless Data Interchange of Vector Data Types](#)

Seamless Data Interchange of Vector Data Types

When the `SQL_TABLE` function runs inside a SPARQL query, vector data types like `REAL_VECTOR` and `HALF_VECTOR` are automatically converted into a user-defined type (UDT) form within SPARQL.

To send the data back to SQL, you need to call the `TO_REAL_VECTOR` or `TO_HALF_VECTOR` SQL functions to insert the data into a SQL table.

For more information, see [TO_REAL_VECTOR Function \(Data Type Conversion\)](#) and [TO_HALF_VECTOR Function \(Data Type Conversion\)](#).

Related Information

[SPARQL_EXECUTE Stored Procedure](#)

[SPARQL_TABLE Function](#)

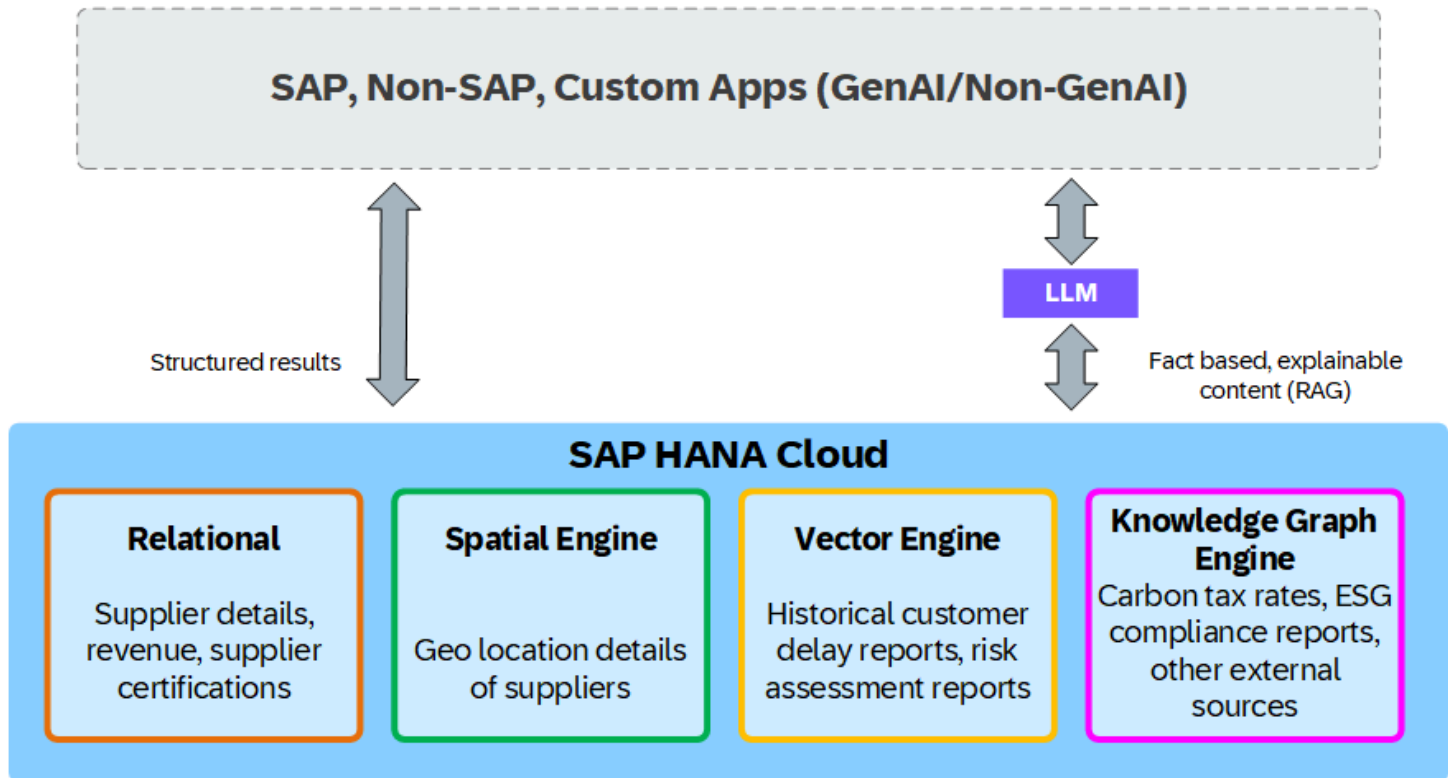
Multi Model Query Example

You can perform a multi model query to find the information that fits your criteria.

In this hybrid Retrieval Augmented Generation (RAG) scenario, we will show you how to use multiple engines within SAP HANA Cloud to find the nearest warehouses in Germany within a 50 km radius of Frankfurt. These warehouses should be for suppliers that are ISO 9001 certified, have low carbon tax rates, and aren't flagged for customs delays.

Multi Model Query:

Find the **nearest warehouse in Germany** for suppliers **with ISO 9001 certification** that **have low carbon tax rates** and **are not flagged for customs delays**.



To achieve this, do the following:

1. Conduct a multi model query to identify the warehouses that meet these criteria. For example, use a SPARQL table within the SAP HANA Cloud knowledge graph engine to filter suppliers that comply with the following conditions: ISO 9001 certified, low carbon tax rates, and not flagged for customs delays.

```
SELECT * FROM SPARQL_TABLE ( '
  SELECT ?supplier ?certification ?carbontax ?flag
  FROM <kg_supplychain>
  WHERE {
    ?supplier a <Supplier> .
    ?supplier <hasCertification> ?certification .
    ?supplier <hasCarbonTaxRate> ?carbontax .
    ?supplier <isFlaggedForDelays> ?flag .
    FILTER(?certification = "ISO 9001" && ?carbontax = "low" && STR (?flag) = "fa
  }
  ');
```

2. Combine the SPARQL_EXECUTE function in the SAP HANA Cloud knowledge graph engine with vector-based semantic filtering and spatial constraints. This helps identify suppliers located within 50 km of Frankfurt whose past customs report narratives indicate no delays.

This hybrid query leverages the SAP HANA Cloud vector engine, the SAP HANA Cloud knowledge graph engine, and the SAP HANA Cloud spatial engine. It ranks nearby suppliers not only by distance but also by their trustworthiness and performance signals.

```
CALL SPARQL_EXECUTE (
```

```
PREFIX ex: <http://example.org/supplier/>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
```

```
SELECT ?uri ?class ?label ?report ?sco
FROM <kg_supplychain>
WHERE {
```

```
    SQL_TABLE ("SELECT \"uri_str\", \"REPORT_TEXT\", \"SCO\"
FROM (SELECT *, SCO - FIRST VALUE (sco) OVER (ORDER BY sco DESC) AS diff
FROM (SELECT \"SUPPLIER_URI\" AS \"uri_str\" \"REPORT_TEXT\", COSINE SIMILARITY (\"RE
AS sco FROM KG SUPPLYCHAIN.SUPPLIER_REPORTS_LOOKUP WHERE \"GEO_LOCATION\".ST_Distance
WHERE diff > -0.2")
BIND(URI (?uri_str) AS ?uri).
BIND(?REPORT_TEXT AS ?report).
BIND(?SCO AS ?sco).
```

```
    ?uri a ?class.
    ?uri <hasCertification> "ISO 9001".
    ?uri <hasCarbonTaxRate> "low".
    ?uri <isFlaggedForDelays> false.
    OPTIONAL { ?uri rdfs:label ?label . }
```

```
}
ORDER BY DESC (?SCO)
LIMIT 10
',
```

```
'Accept: application/sparql-resultset Content-Type: application/sparql-quer
?,
?
```

```
);
```

3. After running your queries, view the best matches for supplier warehouses:

```
un,class, label, report,sco
http://example.org/supplier/AlphaGmbH.Supplier,,Consistently on time. No customs issue.,0.615
http://example.org/supplier/GammaLog/stics.Supplier,, Smooth operations, cleared customs quic
```

Related Information

[SPARQL EXECUTE Stored Procedure](#)

[SPARQL TABLE Function](#)

[SAP HANA Database Vector Engine Guide](#)

[SAP HANA Database Spatial Reference](#)

LangChain

Integrate LangChain with SAP HANA Cloud to make use of vector searches, knowledge graphs, and other in-database capabilities as part of LLM-driven applications.

To use SAP HANA Cloud knowledge graph engine with LangChain, you must install the LangChain SAP HANA Cloud integration package using pip:

```
%pip install -qU langchain-hana
```

To learn more about how to make use of vector searches and knowledge graphs, see [LangChain integration for SAP HANA Cloud](#) ➦ .

You can also use the SAP HANA Cloud knowledge graph engine and SAP HANA Cloud vector engine with LangChain to build a question-answering (QA) chain that queries RDF (Resource Description Framework) data stored in an SAP HANA Cloud instance using the SPARQL query language, which returns a human-readable response. Learn more about [building a QA chain](#) ➦ .

Related Information

[Python Interface](#)

SPARQL Notation Conventions

SPARQL syntax notation conventions.

This reference uses syntax notation similar to BNF (Backus-Naur Form), a notation technique used to define programming languages.

Convention	Description
CAPITALS	Although SPARQL is case-insensitive, SPARQL keywords in this guide are written in UPPERCASE for readability.
[keyword]	Square brackets indicate an optional clause or keyword.
{ }	Curly braces in the SPARQL query WHERE clause are used to define a group of triple patterns that form a basic graph pattern. Example: { ?book dc:creator "John Milton" . ?book dc:title ?title . }
.	A period indicates the end of one triple pattern and the beginning of another: ?book dc:creator "John Milton" .
?	Terms after the question mark are variables that will be filled with the corresponding data from the RDF dataset: SELECT ?person ?email
<i>italics</i>	Terms in italics are variables that you replace in the query.
	Means OR. Indicates that you can use one or more of the specified options.
<uri>	Terms inside angle brackets are URIs (Uniform Resource Identifiers) or prefixes representing URIs.

Related Information

[Terminology](#)

Terminology

SPARQL and RDF terms to be familiar with when working with knowledge graphs.

Term	Definition
DEFAULT graph	SAP Knowledge Graph has a built-in graph called DEFAULT, which is always present and accessible for all users with PUBLIC access privileges – in other words it's readable and writable by all users. This graph is also known as "unnamed graph". Triples stored in the DEFAULT graph can be accessed by specifying ' FROM DEFAULT ' in the SPARQL statements. Any data inserted into SAP Knowledge Graph without specifying a name for the graph is inserted into the DEFAULT graph. Graph names are case-sensitive.
knowledge graph	A knowledge graph is a representation of data, enhanced by real-world information. The knowledge graph captures relationships between entities, such as people, products, or events, and connects them in a meaningful way. A knowledge graph organizes information as triples where facts are represented as subject, predicate, and object. These triples form the core of the knowledge graph, revealing the logical relationships between different entities.
named graph	A graph that was created and named by you or by another user (not to be confused with the DEFAULT graph). Graph names are case-sensitive.
ontology	An ontology is a guiding framework used to define the facts that occur in a domain and how those higher-level descriptions should relate to each other. Instead of focusing on the facts, an ontology regulates the resulting knowledge graph. For example, in the context of movies, an ontology could define the concept of "movies" and its attributes like title, director, genre, and release date, making the framework flexible and scalable for future data. Ontologies act as the blueprint for knowledge graphs, structuring and connecting data to allow deeper insights and reasoning.
plain literal	Plain literals have a lexical form and may include a language tag, as defined in the SPARQL 1.1 Query Language W3C Recommendation 21 March 2013. The language tag is normalized to lowercase.
RAG	Retrieval-augmented generation. A process that supplements the query response of a large language model (LLM) with additional business context not existing within the LLM's static training data. The result aims at minimizing ambiguity and providing answers that align with the user's intention.
RDF	Resource Description Framework. RDF works by representing information using triples, which consist of the following components: • Subject: Represents the resource being described. • Predicate: Represents a property or attribute of the subject. • Object: Represents the value or target of the property. See <i>triple</i> and <i>triple pattern</i>.
RDF graph model	A structure for representing information about resources in the web using nodes and edges.
RDF schema	A framework to describe the structure and types of data in RDF.
SPARQL	SPARQL is a query language designed specifically for querying RDF data. It provides a powerful and expressive syntax for formulating queries that retrieve and manipulate information from RDF knowledge graphs. SELECT

Term	Definition
	Specifies the variables to be returned in the query results. WHERE Defines the pattern or conditions to match in the RDF graph. OPTIONAL Allows for the specification of optional patterns that may or may not be present in the graph. FILTER Allows for the specification of optional patterns that may or may not be present in the graph. GROUP BY Applies conditions on the query variables to further refine the results. ORDER BY Specifies grouping attributes for aggregations. LIMIT or OFFSET Controls the number of results to be returned and allows for pagination. When a SPARQL query is executed, it traverses the RDF graph, matching the specified patterns and applying any conditions or filters. The query engine retrieves the relevant triples that satisfy the query criteria and returns the results in a tabular format.
triple (RDF triple)	A triple in RDF is a data structure that consists of three components: subject, predicate, and object. The subject is the resource being described and can be anything identifiable, such as a webpage, a person, or a book. Predicates describe the relationship between the subject and the object. Objects are values or resources related to the subject through the predicate. All three components are represented by URIs. An example RDF triple: <pre>uri://people#GillesVilleneuve27 http://xmlns.com/foaf/0.1/knows uri://people#AlanJones33 .</pre> An example RDF triple where the prefix ex is defined for the URI <http://example.com/>: <pre>ex:employee42 ex:hireDate ex:state .</pre>
triple pattern	Similar to triple (RDF triple) except that each of the subject, predicate, and object may be a variable (wildcard). <pre>?person <http://xmlns.com/foaf/0.1/name> ?name .</pre> URIs in triple patterns are surrounded by angle brackets.
Turtle	The Turtle data format is a way of representing data using the RDF. It's a way of serializing RDF data in a human-readable and easy-to-write format. Turtle is defined by the W3C and uses a syntax like the N-Triples format, with added support for prefixes and shorthand notations to make it more concise and readable. Turtle is commonly used for writing and sharing RDF data on the web.
typed literal	Typed literals have a lexical form and a data type URI, which is an RDF URI reference.
URI	Uniform Resource Identifier. URIs are web addresses used to identify subjects, predicates, and objects within a graph. For example, <sales> is a prefix representing the full URI <http://sap.com/sales/online>.

Related Information

[SPARQL Notation Conventions](#)

[DEFAULT Graph and Named Graphs](#)